```
//Team Name: BNL
//Team Members: Austin, Nick, Jacob, Makayla, Julia, Ricky, Fred
//Date of last edit, 3/18/26
//Summary: The purpose of this code is for our autonomous rover and it
will have our rover (WALL-E) track a black line and after completing that
task, it will look for a red object and pick it up.
// Include libraries needed for this project
#include <Pixy2.h>  // Allows communication with the Pixy2 camera
#include <Servo.h>  // Controls the servo motor for the claw
#include <CytronMotorDriver.h>  // Controls the left and right drive
motors

// Motor Configuration
CytronMD leftMotor(PWM_DIR, 3, 2);   // Left motor
CytronMD rightMotor(PWM_DIR, 6, 7); // Right motor

// Create objects for Pixy2 camera and claw servo and claw arm servo
Pixy2 pixy;
Servo servoArm;
Servo servoClaw;

//IR Sensor Pins
int IR1 = A1;
int IR2 = A2;
int IR3 = A3;

// Constants (DO NOT CHANGE)
const int CENTER_X = 150;     // The x-position that represents the center
of the camera's view
const int TARGET_WIDTH = 100; // The object's width when it is close
enough to grab
const float Kp = 6.5;         // Proportional gain for turning control
(used for feedback)
const int MAX_SPEED = 130;    // Maximum motor speed
const int MIN_SPEED = 30;     // Minimum motor speed (ensures the robot
moves instead of stalling)
const int FORWARD_SPEED = 200; // Speed for moving forward when
approaching the object
const int TURN_THRESHOLD = 8; // How close the object must be to the
center before moving forward
//done represents a flag that will signal to our rover when it has
completed the course
int done = 0;
void setup() {
    Serial.begin(115200);
    servoArm.attach(10);
    servoClaw.attach(5);  // Attach the servo motor to digital pin 5
```

```
    servoArm.write(115);
    servoClaw.write(135);
    pixy.init();     // Initialize Pixy2 camera

  //IR Sensor Setup
  pinMode(IR1, INPUT);
  pinMode(IR2, INPUT);
  pinMode(IR3, INPUT);
  //wait to start
  delay(5000);
}

void loop() {
//read the IR Sensors
int IRValue1 = digitalRead(IR1);
int IRValue2 = digitalRead(IR2);
int IRValue3 = digitalRead(IR3);

//Left
if (done==0)
{
if (IRValue1 == 1 && IRValue2 == 1 && IRValue3 == 0){
leftMotor.setSpeed(-140);
rightMotor.setSpeed(-255);
}
//Sharp Left
else if (IRValue1 == 1 && IRValue2 == 0 && IRValue3 == 0){
leftMotor.setSpeed(-85);
rightMotor.setSpeed(-255);
}
//straight
else if (IRValue1 == 0 && IRValue2 == 1 && IRValue3 == 0){
leftMotor.setSpeed(-255);
rightMotor.setSpeed(-255);
}
//Right
else if (IRValue1 == 0 && IRValue2 == 1 && IRValue3 == 1){
leftMotor.setSpeed(-255);
rightMotor.setSpeed(-140);
}
//Sharp Right
else if (IRValue1 == 0 && IRValue2 == 0 && IRValue3 == 1){
leftMotor.setSpeed(-255);
rightMotor.setSpeed(-85);
}
else if (IRValue1 == 0 && IRValue2 == 0 && IRValue3 == 0){
 /* if (turn==-1)
```

```
  {
    motor1.setSpeed(-75);
    motor2.setSpeed(-255);
  }
  else if (turn==0)
  {
    motor1.setSpeed(-255);
    motor2.setSpeed(-255);
  }
  else if (turn==1)
  {
    motor1.setSpeed(-255);
    motor2.setSpeed(-75);
  }

}
else if (IRValue1 == 1 && IRValue2 == 1 && IRValue3 == 1){
motor1.setSpeed(-10);
motor2.setSpeed(-10);
}
/
else if (IRValue1 == 0 && IRValue2 == 1&& IRValue3 == 0){
*/
}
else if (IRValue1 == 1 && IRValue2 == 1 && IRValue3 == 1)
{
  trackObject();
  done = 1;
}
}
else
{
  trackObject();
}
}
/*****************************************************
 * FUNCTION DEFINITIONS
 *****************************************************/

// Function to track and approach an object using Pixy2
void trackObject() {

    // Retrieve detected objects from Pixy2
    pixy.ccc.getBlocks();
    // Check if an object is detected
    if (pixy.ccc.numBlocks > 0){
      //If an object is detected:
```

```
      //   - Get the object's x-coordinate (m_x) and width (m_width)
      int object_x = pixy.ccc.blocks[0].m_x;
      int object_width = pixy.ccc.blocks[0].m_width;
      //   - Compute the error between object_x and CENTER_X
      int error = object_x - CENTER_X;
      //   - Calculate turn speed using proportional control: abs(error) *
Kp
      int error_speed = abs(error)*Kp;
      //   - Constrain the turn speed between MIN_SPEED and MAX_SPEED
      int turn_speed = constrain(error_speed, MIN_SPEED, MAX_SPEED);
      // Controlling the rover until it is within our desired reach of the
object, then it will pick the can up
if(object_width >= TARGET_WIDTH){
          grabObject();
          delay(20000);
        }
      else if(error < -TURN_THRESHOLD)
      {
        Serial.println("1");
        turnLeft(turn_speed);
      }
      else if(error > TURN_THRESHOLD){
        Serial.println("2");
        turnRight(turn_speed);
      }
      else //if(error == 0)
      {
        Serial.println("3");
        moveForward(FORWARD_SPEED);
        /*if(object_width >= TARGET_WIDTH){
          grabObject();
        }
        else{
          moveForward(FORWARD_SPEED);
        }*/
      }
    }
    else
    {
      leftMotor.setSpeed(255);
      rightMotor.setSpeed(-255);
    }
}

// Function to stop the robot and close the claw
void grabObject() {
    //Stop the motors before grabbing the object
```

```
    stopMotors();
    //Close Claw and set arm to correct position
    servoArm.write(120);
    servoClaw.write(90);
    //Slight delay
    delay(500);
    // Raise arm up with the cann in the claw
    servoArm.write(145);
    delay(1000);
}

// Function to move forward
void moveForward(int speed) {
    leftMotor.setSpeed(-speed);
    rightMotor.setSpeed(-speed);
}

// Function to turn right
void turnRight(int speed) {
    int x = FORWARD_SPEED - speed;
    leftMotor.setSpeed(-FORWARD_SPEED);
    rightMotor.setSpeed(-x);
}

// Function to turn left
void turnLeft(int speed) {
    int x = FORWARD_SPEED - speed;
    leftMotor.setSpeed(-x);
    rightMotor.setSpeed(-FORWARD_SPEED);
}

// Function to stop motors
void stopMotors() {
    leftMotor.setSpeed(0);
    rightMotor.setSpeed(0);
}
```